

Shashwath Ks Assignment 3 Day 03

Programming Essentials For Ai

Dated : 18/04/2026

Source-W3Schools

Topic
Python If...Else
Python Match
Python While Loops
Python For Loops
Python Functions
NumPy
Pandas

Python If Statement

Python Conditions and If statements

Python supports the usual logical conditions from mathematics:

- Equals: `a == b`
- Not Equals: `a != b`
- Less than: `a < b`
- Less than or equal to: `a <= b`
- Greater than: `a > b`
- Greater than or equal to: `a >= b`

These conditions can be used in several ways, most commonly in "if statements" and loops.

An "if statement" is written by using the if keyword.

```
# Example of If statement

a = 33
b = 200
if b > a: # Check whether the value of b is greater than a
    print("b is greater than a") # prints only if the above condition is True
```

Output Block

b is greater than a

In this example we use two variables, `a` and `b`, which are used as part of the if statement to test whether `b` is greater than `a`. As `a` is `33`, and `b` is `200`, we know that `200` is greater than `33`, and so we print to screen that `"b is greater than a"`.

How If Statements Work

The if statement evaluates a condition (an expression that results in `True` or `False`). If the condition is true, the code block inside the if statement is executed. If the condition is false, the code block is skipped.

```
# Checking if a number is positive:
number = 15
if number > 0: # condition to check the number is greater than 0
    print("The number is positive") # returns this block if the above statement is True.
```

Output Block

```
The number is positive
```

Indentation

Python uses indentation (spaces or tabs at the beginning of a line) to determine the structure and scope of code blocks. Unlike many other programming languages that rely on curly braces to group statements, Python enforces consistent indentation as part of its syntax. This makes the code more readable and visually organized while clearly showing the relationship between different parts of the program.

```
# If statement, without indentation (will raise an error):

a = 33
b = 200
if b > a: # The following line has no indentation so the if statement has no following
block to execute
# There is no `pass` defined either, so this will return an error`
print("b is greater than a") # you will get an error
```

Output Block

```
IndentationError: expected an indented block after 'if' statement on line 5
(1050164311.py, line 7)
```

Multiple Statements in If Block

In Python, an if block can contain several statements that are executed when the condition evaluates to true. All of these statements must be indented consistently at the same level to indicate they belong to the same block. Proper indentation ensures clarity and helps Python correctly interpret which lines are part of the conditional structure.

```
# Multiple statements in an if block

age = 20
```

```
if age >= 18:
    '''Only enters this block of code if and only if the condition is True'''
    print("You are an adult")
    print("You can vote")
    print("You have full legal rights")
```

Output Block

```
You are an adult
You can vote
You have full legal rights
```

Using Variables in Conditions

In Python, boolean variables can be used directly in if statements without needing explicit comparison operators. If the variable evaluates to True, the block of code inside the condition will run. This simplifies the syntax and makes the code cleaner and easier to read while still being clear in its intent.

```
# Using a boolean variable:

is_logged_in = True # Will be Set to True
if is_logged_in: # Condition is evaluated as True
    print("Welcome back!") # Block will be executed
```

Output Block

```
Welcome back!
```

Python can evaluate many types of values as **True** or **False** in an if statement.

- Zero (0), empty strings (""), None, and empty collections are treated as False. Everything else is treated as True.
- This includes positive numbers (5), negative numbers (-3), and any non-empty string (even "False" is treated as True because it's a non-empty string).

Python Elif Statement

The Elif Keyword

The **elif** keyword is Python's way of saying "if the previous conditions were not true, then try this condition".

The **elif** keyword allows you to check multiple expressions for True and execute a block of code as soon as one of the conditions evaluates to True.

In this example a is equal to b, so the first condition is not true, but the elif condition is true, so we print to screen that "a and b are equal".

```
a = 33
b = 33
```

```
if b > a: # Tests whether b is greater than a at first
    print("b is greater than a")
elif a == b: # If the initial condition is not True then it passes on to the elif
    print("a and b are equal")
```

Output Block

a and b are equal

Multiple Elif Statements

You can have as many `elif` statements as you need. Python will check each condition in order and execute the first one that is true.

In this example, the program checks each condition in order. Since score is 75, it prints "Grade: C" (the first condition that evaluates to true).

```
# Multiple elif statements to check and classify the grade based on the score
score = 75
# Will evaluate the score sequentially if the initial condition is not True then it
moves on to the next
if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
elif score >= 60:
    print("Grade: D")
```

Output Block

Grade: C

How Elif Works

When you use `elif`, Python evaluates the conditions from top to bottom. As soon as it finds a condition that is true, it executes that block and skips all remaining conditions.

Important: Only the first true condition will be executed. Even if multiple conditions are true, Python stops after executing the first matching block.

```
# The following program is to categorizing age groups:

age = 25

if age < 13:
    print("You are a child")
elif age < 20:
    print("You are a teenager")
elif age < 65:
```

```
print("You are an adult")
elif age >= 65:
    print("You are a senior")
```

Output Block

You are an adult

When to Use Elif

Use `elif` when you have multiple mutually exclusive conditions to check. This is more efficient than using multiple separate `if` statements because Python stops checking once it finds a true condition.

```
# Day of the week checker

day = 3
# Similar evaluation like what we have seen in programs above
if day == 1:
    print("Monday")
elif day == 2:
    print("Tuesday")
elif day == 3:
    print("Wednesday")
elif day == 4:
    print("Thursday")
elif day == 5:
    print("Friday")
elif day == 6:
    print("Saturday")
elif day == 7:
    print("Sunday")
# Does not have a fallback condition else for other invalid input type
```

Output Block

Wednesday

Python Else Statement

The Else Keyword

The `else` keyword catches anything which is not caught by the preceding conditions.

The `else` statement is executed when the `if` condition and any `elif` conditions evaluate to False.

```
a = 200
b = 33
if b > a: # Checks this condition first
    print("b is greater than a")
elif a == b: # Checks this condition if the first condition is not True
    print("a and b are equal")
```

```
else: # If all condtions above are not True then this block is executed.  
    print("a is greater than b")
```

Output Block

```
a is greater than b
```

In this example a is greater than b, so the first condition is not true, also the elif condition is not true, so we go to the else condition and print to screen that "a is greater than b".

Else Without Elif

You can also have an `else` without the `elif`

```
# Using if-else statement  
a = 200  
b = 33  
if b > a: # First condtion evaluation  
    print("b is greater than a")  
else: # Fallback for if condition when evaluated False  
    print("b is not greater than a")
```

Output Block

```
b is not greater than a
```

How Else Works

The `else` statement provides a default action when none of the previous conditions are true. Think of it as a "catch all" for any scenario not covered by your `if` and `elif` statements.

Note: The `else` statement must come last. You cannot have an `elif` after an `else`.

```
# Checking even or odd numbers  
  
number = 7  
  
if number % 2 == 0: # Using modulus operator in if to evaluate the condtion  
    print("The number is even")  
else:  
    print("The number is odd")
```

Output Block

```
The number is odd
```

Complete If-Elif-Else Chain

You can combine if, elif, and else to create a comprehensive decision-making structure.

```
# Temperature classifier:

temperature = 22
# Using if-elif-else to create a comprehensive decision making structure.
if temperature > 30:
    print("It's hot outside!")
elif temperature > 20:
    print("It's warm outside")
elif temperature > 10:
    print("It's cool outside")
else:
    print("It's cold outside!")
```

Else as Fallback

The else statement acts as a fallback that executes when none of the preceding conditions are true. This makes it useful for error handling, validation, and providing default values.

```
# Validating user input

username = "Emil"
# Check whether the Username is empty
if len(username) > 0:
    print(f"Welcome, {username}!") #greet whether the username exists
else:
    print("Error: Username cannot be empty") # fallback if username is empty
```

Output Block

```
Welcome, Emil!
```

Python Shorthand If

Short Hand If

If you have only one statement to execute, you can put it on the same line as the `if` statement.

```
# One-line if statement:
a = 5
b = 2
# This is the shorthand form of if in a single line
if a > b: print("a is greater than b")
```

Output Block

```
a is greater than b
```

Short Hand If ... Else

If you have one statement for `if` and one for `else`, you can put them on the same line using a conditional expression

```
# One-line if/else that prints a value
a = 2
b = 330
print("A") if a > b else print("B")
```

Output Block

B

Assign a Value With If ... Else

You can also use a one-line if/else to choose a value and assign it to a variable.

```
a = 10
b = 20
bigger = a if a > b else b # Assigning the value of a variable based on if condition
evaluation
print("Bigger is", bigger)
```

Output Block

Bigger is 20

The syntax follows this pattern:

variable = value_if_true if condition else value_if_false

Multiple Conditions on One Line

You can chain conditional expressions, but keep it short so it stays readable.

```
# One line, three outcomes:

a = 330
b = 330
# Prints the statement as it evaluates and returns the result
print("A") if a > b else print("=") if a == b else print("B")
```

Output Block

=

Practical Examples

Ternary operators are particularly useful for simple assignments and return statements.


```
# Finding the maximum of two numbers:
```

```
x = 15  
y = 20  
max_value = x if x > y else y  
print("Maximum value:", max_value)
```

Output Block

```
Maximum value: 20
```

```
# Setting a default value
```

```
username = "" # Username is not set  
display_name = username if username else "Guest" # Default will be set to guest if  
username is empty  
print("Welcome,", display_name)
```

Output Block

```
Welcome, Guest
```

When to Use Shorthand If

Shorthand `if` statements and ternary operators should be used when:

- The condition and actions are simple
- It improves code readability
- You want to make a quick assignment based on a condition

Python Logical Operators

Logical operators are used to combine conditional statements. Python has three logical operators:

- `and` - Returns True if both statements are true
- `or` - Returns True if one of the statements is true
- `not` - Reverses the result, returns False if the result is true.

The and Operator

The `and` keyword is a logical operator, and is used to combine conditional statements. Both conditions must be true for the entire expression to be true.

```
# Test if a is greater than b, AND if c is greater than a:
```

```
a = 200  
b = 33
```

```
c = 500
if a > b and c > a: # Two conditions evaluated in one-line using `and`
    print("Both conditions are True")
```

Output Block

```
Both conditions are True
```

The or Operator

The or keyword is a logical operator, and is used to combine conditional statements. At least one condition must be true for the entire expression to be true.

```
# Test if a is greater than b, OR if a is greater than c:

a = 200
b = 33
c = 500
if a > b or a > c: # Tests two conditions will return if either one of it is True
    print("At least one of the conditions is True")
```

Output Block

```
At least one of the conditions is True
```

The not Operator

The not keyword is a logical operator, and is used to reverse the result of the conditional statement.

```
# Test if a is NOT greater than b:

a = 33
b = 200
if not a > b: # Checks the negation of the evaluated expression
    print("a is NOT greater than b")
```

Output Block

```
a is NOT greater than b
```

Combining Multiple Operators

You can combine multiple logical operators in a single expression. Python evaluates **not** first, then **and**, then **or**.

```
# Combining and, or, and not:

age = 25
```

```
is_student = False
has_discount_code = True
# Checks multiple conditions with mix of and, or
if (age < 18 or age > 65) and not is_student or has_discount_code:
    print("Discount applies!")
```

Output Block

Discount applies!

Truth Tables

Understanding how logical operators work with different values

and Operator Truth Table

Condition 1	Condition 2	Result
True	True	True
True	False	False
False	True	False
False	False	False

or Operator Truth Table

Condition 1	Condition 2	Result
True	True	True
True	False	True
False	True	True
False	False	False

Using Parentheses for Clarity

When combining multiple logical operators, use parentheses to make your intentions clear and control the order of evaluation.

```
# Using parentheses for complex conditions:

temperature = 25
is_raining = False
is_weekend = True
# Paranthesis is added for keeping the evaluations within the paranthesis and evaluate
that expression with the one outside.
if (temperature > 20 and not is_raining) or is_weekend:
    print("Great day for outdoor activities!")
```

Output Block

Great day for outdoor activities!

```
# User authentication check:

username = "Tobias"
password = "secret123"
is_verified = True
# User validation with multiple and parmaters
# only executes the block of code if all condtions are True
if username and password and is_verified:
    print("Login successful")
else:
    print("Login failed")
```

Output Block

Login successful

```
# Range checking with logical operators

score = 85
# Checking the validity of the score if it is within the range
if score >= 0 and score <= 100:
    print("Valid score")
else:
    print("Invalid score")
```

Output Block

Valid score

Python Nested If

Nested If Statements

You can have `if` statements inside `if` statements. This is called nested `if` statements.

```
# Example of nesting if within an If condtion
x = 41

if x > 10:
    print("Above ten,")
    if x > 20:
        print("and also above 20!")
    else:
        print("but not above 20.")
```

Output Block

Above ten,
and also above 20!

How Nested If Works

Each level of nesting creates a deeper level of decision making. The code evaluates from the outermost condition inward.

```
# Checking multiple conditions with nesting:

age = 25
has_license = True
# Evaluating nested conditions
if age >= 18:
    if has_license: # expression evaluated if above is True
        print("You can drive")
    else:
        print("You need a license") # fallback code within nested condition
else:
    print("You are too young to drive") # Fallback for the main
```

Output Block

You can drive

Multiple Levels of Nesting

You can nest as many levels deep as needed, but keep in mind that too many levels can make code harder to read.

```
# Three levels of nesting

score = 85
attendance = 90
submitted = True

if score >= 60: # first check: minimum passing score
    if attendance >= 80: # second check: required attendance
        if submitted: # third check: assignment submitted
            print("Pass with good standing") # all conditions met
        else:
            print("Pass but missing assignment") # no submission
    else:
        print("Pass but low attendance") # attendance not sufficient
else:
    print("Fail") # score too low
```

Output Block

Pass with good standing

Nested If vs Logical Operators

Sometimes nested if statements can be simplified using logical operators like **and**. The choice depends on your logic.

```
# This nested if:

temperature = 25
is_sunny = True

if temperature > 20:
    if is_sunny:
        print("Perfect beach weather!")

# Could also be written with and

temperature = 25
is_sunny = True

if temperature > 20 and is_sunny:
    print("Perfect beach weather!")
```

Output Block

```
Perfect beach weather!
Perfect beach weather!
```

Both approaches produce the same result. Use nested if statements when the inner logic is complex or depends on the outer condition. Use **and** when both conditions are simple and equally important.

```
# Login validation with nested checks

username = "Emil"
password = "python123"
is_active = True

if username: # check if username is provided
    if password: # check if password is provided
        if is_active: # check if account is active
            print("Login successful") # all conditions satisfied
        else:
            print("Account is not active") # user exists but inactive
    else:
        print("Password required") # missing password
else:
    print("Username required") # missing username
```

Output Block

```
Login successful
```

```
# Grade calculation with nested logic

score = 92
extra_credit = 5

if score >= 90:
    if extra_credit > 0:
        print("A+ grade")
    else:
        print("A grade")
elif score >= 80:
    print("B grade")
else:
    print("C grade or below")
```

Output Block

A+ grade

Python Pass Statement

The Pass Statement

`if` statements cannot be empty, but if you for some reason have an `if` statement with no content, use the `pass` statement to avoid getting an error.

```
a = 33
b = 200
# Condition is checked
if b > a:
    pass # nothing is returned but will not return an Error
```

The `pass` statement is a null operation - nothing happens when it executes. It serves as a placeholder.

Why Use `pass`?

The `pass` statement is useful in several situations:

- When you're creating code structure but haven't implemented the logic yet
- When a statement is required syntactically but no action is needed
- As a placeholder for future code during development
- In empty functions or classes that you plan to implement later

`pass` in Development

During development, you might want to sketch out your program structure before implementing the details. The `pass` statement allows you to do this without syntax errors.

```
# Placeholder for future implementation:

age = 16
```

```
if age < 18:
    pass # TODO: Add underage logic later
else:
    print("Access granted")
```

pass vs Comments

- A comment is ignored by Python
- `pass` is an actual statement that gets executed, though it does nothing
- You need `pass` where Python expects a statement, not just a comment
- Comments are used to explain code, while `pass` is used to satisfy Python's syntax requirements
- Using `pass` prevents errors in empty blocks like `if`, `for`, `while`, functions, or classes
- Comments cannot be used as placeholders in executable blocks because Python completely ignores them

```
score = 85

if score > 90:
    # This is excellent
# This will raise an IndentationError due to absence of pass
```

```
# This works correctly with pass:
```

```
score = 85

if score > 90:
    pass # This is excellent
print("Score processed")
```

Output Block

```
Score processed
```

pass with Multiple Conditions

You can use `pass` in any branch of an if-elif-else statement.

```
## Using pass in different branches:

value = 50

if value < 0:
    print("Negative value")
elif value == 0:
    pass # Zero case - no action needed
```



```
else:
    print("Positive value")
```

Output Block

Positive value

pass in Other Contexts

While we focus on pass with if statements here, it's also commonly used with loops, functions, and classes.

```
# Using pass with functions:

def calculate_discount(price):
    pass # TODO: Implement discount logic

# Function exists but doesn't do anything yet
```

Python Match

The match Statement

The `match` statement is used to perform different actions based on different conditions.

The Python Match Statement

Instead of writing many `if..else` statements, you can use the `match` statement.

The `match` statement selects one of many code blocks to be executed.

Syntax:

```
match expression:
    case x:
        code block
    case y:
        code block
    case z:
        code block
```

This is how it works:

- The match expression is evaluated once.
- The value of the expression is compared with the values of each case.
- If there is a match, the associated block of code is executed. The example below uses the weekday number to print the weekday name.

```

day = 4

match day: # check value of day using pattern matching
    case 1:
        print("Monday") # day 1
    case 2:
        print("Tuesday") # day 2
    case 3:
        print("Wednesday") # day 3
    case 4:
        print("Thursday") # day 4
    case 5:
        print("Friday") # day 5
    case 6:
        print("Saturday") # day 6
    case 7:
        print("Sunday") # day 7

```

Output Block

Thursday

Default Value

Use the underscore character `_` as the last case value if you want a code block to execute when there are not other matches:

```

day = 4

match day: # match the value of day
    case 6:
        print("Today is Saturday") # Saturday case
    case 7:
        print("Today is Sunday") # Sunday case
    case _:
        print("Looking forward to the Weekend") # default case if no match

```

Output Block

Looking forward to the Weekend

Combine Values

Use the pipe character `|` as an or operator in the case evaluation to check for more than one value match in one case.

```

day = 4

match day: # check value of day using pattern matching
    case 1 | 2 | 3 | 4 | 5: # multiple values for weekdays
        print("Today is a weekday") # Monday to Friday
    case 6 | 7: # weekend values

```

```
print("I love weekends!") # Saturday and Sunday
```

Output Block

Today is a weekday

If Statements as Guards

You can add if statements in the case evaluation as an extra condition-check.

```
month = 5
day = 4

match day: # match based on day value
    case 1 | 2 | 3 | 4 | 5 if month == 4: # weekday in April
        print("A weekday in April")
    case 1 | 2 | 3 | 4 | 5 if month == 5: # weekday in May
        print("A weekday in May")
    case _: # default case if no match
        print("No match")
```

Output Block

A weekday in May

Python While Loops

Python Loops

Python has two primitive loop commands:

- `while` loops
- `for` loops

The while Loop

With the `while` loop we can execute a set of statements as long as a condition is true.

```
# Print i as long as i is less than 6:

i = 1
while i < 6: # Checks condition and exits the loop when i becomes 6
    print(i)
    i += 1
```

Output Block

```
1
2
3
4
5
```

The while loop requires relevant variables to be ready, in this example we need to define an indexing variable, *i*, which we set to 1.

The break Statement

With the break statement we can stop the loop even if the while condition is true.

```
# Exit the loop when i is 3:

i = 1
while i < 6: # Loop is set to execute until i is less than 6
    print(i)
    if i == 3: # checks if i is 3
        break # exits the loop without continuing to iterate further
    i += 1
```

Output Block

```
1
2
3
```

The continue Statement

With the continue statement we can stop the current iteration, and continue with the next.

```
# Continue to the next iteration if i is 3:

i = 0
while i < 6:
    i += 1 # pre-increment
    if i == 3: # If i becomes 3, then the iteration is skipped
        continue # continue skips the current iteration
    print(i)
```

Output Block

```
1
2
4
5
6
```

The else Statement

With the `else` statement we can run a block of code once when the condition no longer is true.

```
i = 1

while i < 6: # loop runs while condition is True
    print(i) # print current value of i
    i += 1 # increment i by 1
else:
    print("i is no longer less than 6") # runs when loop condition becomes False
```

Output Block

```
1
2
3
4
5
i is no longer less than 6
```

```
# Code challenge for while-loops
# Create the i variable
i = 0
# While loop: print 1-5, skip 3 with continue
while i < 6:
    i+=1
    if i == 3:
        continue
    print(i)
```

Output Block

```
1
2
4
5
6
```

Python For Loops

A **for** loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).

This is less like the **for** keyword in other programming languages, and works more like an iterator method as found in other object oriented programming languages.

With the **for** loop we can execute a set of statements, once for each item in a list, tuple, set etc.

```
# Print each fruit in a fruit list:

fruits = ["apple", "banana", "cherry"]
for x in fruits: # prints the contents in the loop one-by-one
    print(x)
```

The `for` loop does not require an indexing variable to set beforehand.

Looping Through a String

Even strings are iterable objects, they contain a sequence of characters.

```
# Loop through the letters in the word "banana":  
  
for x in "banana":  
    print(x) #prints the string character one after another
```

Output Block

```
b  
a  
n  
a  
n  
a
```

The break Statement

With the `break` statement we can stop the loop before it has looped through all the items.

```
# Exit the loop when x is "banana":  
  
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x) # Fruits variable printed one after another  
    if x == "banana": # but when the item is 'banana' it prints and then breaks the loop  
        and exits  
        break
```

Output Block

```
apple  
banana
```

```
# Exit the loop when x is "banana", but this time the break comes before the print:  
  
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    if x == "banana": # Here the banana is not printed the loop exits  
        break  
    print(x)
```

Output Block

```
apple
```

The continue Statement

With the continue statement we can stop the current iteration of the loop, and continue with the next:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    if x == "banana": # Skips the iteration of banana item in the list
        continue
    print(x)
```

Output Block

```
apple
cherry
```

The range() Function

To loop through a set of code a specified number of times, we can use the `range()` function.

The `range()` function returns a sequence of numbers, starting from 0 by default, and increments by 1 by default, and ends at a specified number.

The `range()` function defaults to 0 as a starting value, however it is possible to specify the starting value by adding a parameter: `range(2, 6)`, which means values from 2 to 6 (but not including 6):

```
# Using the range() function:

for x in range(6): #prints from 0 as start to 5 end
    print(x, end='')
print("\n", "-"*5)
# Using the start parameter:

for x in range(2, 6): # starts from 2 ends at 5
    print(x, end="")
```

Output Block

```
012345
-----
2345
```

The `range()` function defaults to increment the sequence by 1, however it is possible to specify the increment value by adding a third parameter: `range(2, 30, 3)`:

```
# Increment the sequence with 3 (default is 1):

for x in range(2, 30, 3): #defines the start end and the number of the skips
    print(x, "\t", end = "")
```

Output Block

2 5 8 11 14 17 20 23 26 29

Else in For Loop

The else keyword in a for loop specifies a block of code to be executed when the loop is finished.

```
for x in range(6): # loop from 0 to 5
    print(x) # print current number
else:
    print("Finally finished!") # runs after loop completes normally
```

Output Block

```
0
1
2
3
4
5
Finally finished!
```

```
# Break the loop when x is 3, and see what happens with the else block:

for x in range(6):
    if x == 3: break # breaks the loop at x == 3
    print(x)
else:
    print("Finally finished!") # else is not executed if it encounters break
```

Output Block

```
0
1
2
```

Nested Loops

A nested loop is a loop inside a loop.

The "inner loop" will be executed one time for each iteration of the "outer loop".

```
adj = ["red", "big", "tasty"]
fruits = ["apple", "banana", "cherry"]

for x in adj: # loop through adjectives
    for y in fruits: # loop through fruits for each adjective
        print(x, y) # print combination of adjective and fruit
```

Output Block


```
red apple
red banana
red cherry
big apple
big banana
big cherry
tasty apple
tasty banana
tasty cherry
```

The pass Statement

for loops cannot be empty, but if you for some reason have a for loop with no content, put in the pass statement to avoid getting an error.

```
for x in [0, 1, 2]:
    pass # Placeholder defined so the indentation is defined also so that it does not
        throw an exception
```

Python Functions

- A function is a block of code which only runs when it is called.
- A function can return data as a result.
- A function helps avoid code repetition.

Creating a Function

In Python, a function is defined using the `def` keyword, followed by a function name and parentheses.

This creates a function named `my_function` that prints "Hello from a function" when called.

```
def my_function():
    print("Hello from a function") # Defines the function and returns a print statement

my_function() # Calling a function
```

Output Block

```
Hello from a function
```

```
# We can call the function multiple times.
my_function()
my_function()
my_function()
```

Output Block

```
Hello from a function
Hello from a function
Hello from a function
```

Function Names

Function names follow the same rules as variable names in Python:

- A function name must start with a letter or underscore
- A function name can only contain letters, numbers, and underscores
- Function names are case sensitive so `myFunction` and `myfunction` are different

Valid **function** names:

```
calculate_sum()
_private_function()
myFunction2()
```

Why Use Functions?

Imagine you need to convert temperatures from Fahrenheit to Celsius several times in your program. Without functions, you would have to write the same calculation code repeatedly.

```
temp1 = 77
celsius1 = (temp1 - 32) * 5 / 9 # convert Fahrenheit to Celsius
print(celsius1) # print result

temp2 = 95
celsius2 = (temp2 - 32) * 5 / 9 # conversion again (repeated code)
print(celsius2)

temp3 = 50
celsius3 = (temp3 - 32) * 5 / 9 # same calculation repeated
print(celsius3)
```

Output Block

```
25.0
35.0
10.0
```

With functions, you write the code once and reuse it

```
# Function to convert fahrenheit to celsius
def fahrenheit_to_celsius(fahrenheit): # Function with the param of temp in fahrenheit
    return (fahrenheit - 32) * 5 / 9
# Can be used multiple times for different values
print(fahrenheit_to_celsius(77))
```

```
print(fahrenheit_to_celsius(95))
print(fahrenheit_to_celsius(50))
```

Output Block

```
25.0
35.0
10.0
```

Return Values

Functions can send data back to the code that called them using the return statement.

When a function reaches a return statement, it stops executing and sends the result back.

```
# Function defined to make the greeting
def get_greeting():
    return "Hello from a function"
# variable assigned with the function result
message = get_greeting()
print(message)
```

Output Block

```
Hello from a function
```

```
# print the return value from the function directly
def get_greeting():
    return "Hello from a function"
# print the function return values directly
print(get_greeting())
```

Output Block

```
Hello from a function
```

The pass Statement

Function definitions cannot be empty. If you need to create a function placeholder without any code, use the pass statement.

```
def my_function():
    pass # Defining the empty the function
```

The pass statement is often used when developing, allowing you to define the structure first and implement details later.

Python Function Arguments

Arguments

Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

The following example has a function with one argument (**fname**). When the function is called, we pass along a first name, which is used inside the function to print the full name.

```
# A function with one argument:

def my_function(fname): # Function with only one arg
    print(fname + " Smith")
# calling the same function 3 times of different values
my_function("Emil")
my_function("Tobias")
my_function("Linus")
```

Output Block

```
Emil Smith
Tobias Smith
Linus Smith
```

Parameters vs Arguments

The terms parameter and argument can be used for the same thing: information that are passed into a function.

```
def my_function(name): # name is a parameter
    print("Hello", name)

my_function("Emil") # "Emil" is an argument
```

Output Block

```
Hello Emil
```

Number of Arguments

By default, a function must be called with the correct number of arguments.

If your function expects 2 arguments, you must call it with exactly 2 arguments.

```
# This function expects 2 arguments, and gets 2 arguments::

def my_function(fname, lname):
    print(fname + " " + lname)
```

```
my_function("Emil", "Smith")
```

Output Block

```
Emil Smith
```

If you try to call the function with the wrong number of arguments, you will get an error

Default Parameter Values

You can assign default values to parameters. If the function is called without an argument, it uses the default value.

```
def my_function(name = "friend"): # default value defined if the parameter is empty
    print("Hello", name)

# Following parameters have values explicitly mentioned
my_function("Emil")
my_function("Tobias")
my_function("Linus")
my_function() # empty parameter
```

Output Block

```
Hello Emil
Hello Tobias
Hello Linus
Hello friend
```

```
def my_function(country = "Norway"): # default parameter value is Norway
    print("I am from", country) # print country name

my_function("Sweden") # passes Sweden
my_function("India") # passes India
my_function() # uses default value (Norway)
my_function("Brazil") # passes Brazil
```

Output Block

```
I am from Sweden
I am from India
I am from Norway
I am from Brazil
```

Keyword Arguments

You can send arguments with the key = value syntax.

```
def my_function(animal, name):
    print("I have a", animal)
    print("My", animal + "'s name is", name)

my_function(animal = "dog", name = "Buddy") # Assigning values while calling the
function

# This way, with keyword arguments, the order of the arguments does not matter.

my_function(name = "Buddy", animal = "dog")
```

Output Block

```
I have a dog
My dog's name is Buddy
I have a dog
My dog's name is Buddy
```

Positional Arguments

When you call a function with arguments without using keywords, they are called positional arguments.

Positional arguments must be in the correct order.

```
def my_function(animal, name): # order of param is (animal, name)
    print("I have a", animal)
    print("My", animal + "'s name is", name)
# parameters are defined by the positions where it follows the order defined in the
function
my_function("dog", "Buddy")

# The order matters with positional arguments

my_function("Buddy", "dog") # Example of why it matters to follow the order of the
param
```

Output Block

```
I have a dog
My dog's name is Buddy
I have a Buddy
My Buddy's name is dog
```

Mixing Positional and Keyword Arguments

You can mix positional and keyword arguments in a function call.

However, positional arguments must come before keyword arguments.

```
# function with the order postioned parameters
def my_function(animal, name, age):
    print("I have a", age, "year old", animal, "named", name)
# function call with mixed arguments
```

```
my_function("dog", name = "Buddy", age = 5)
```

Output Block

I have a 5 year old dog named Buddy

Passing Different Data Types

You can send any data type as an argument to a function (string, number, list, dictionary, etc.).

The data type will be preserved inside the function.

```
# Sending a list as an argument:
def my_function(fruits):
    for fruit in fruits:
        print(fruit)
# Defining the list
my_fruits = ["apple", "banana", "cherry"]
my_function(my_fruits) # sending list as an argument
```

Output Block

apple
banana
cherry

```
# Function which extracts the information from the dictionary

def my_function(person):
    print("Name:", person["name"])
    print("Age:", person["age"])
#Sending a dictionary as an argument:
my_person = {"name": "Emil", "age": 25}
my_function(my_person)
```

Output Block

Name: Emil
Age: 25

Return Values

Functions can return values using the return statement.

```
def my_function(x, y):
    return x + y # Returns the sum of the two variables x and y

result = my_function(5, 3) # my function returns the result only stored in a var
print(result)
```

Returning Different Data Types

Functions can return any data type, including lists, tuples, dictionaries, and more.

```
# A function that returns a list

def my_function():
    # List defined in a function
    return ["apple", "banana", "cherry"]

fruits = my_function()
# Explicitly defining to return the item in the list index defined in a function
print(fruits[0])
print(fruits[1])
print(fruits[2])
```

Output Block

```
apple
banana
cherry
```

```
# A function that returns a tuple
def my_function():
    return (10, 20) # function returns a tuple

x, y = my_function() # unpack returned tuple into x and y
print("x:", x) # print first value
print("y:", y) # print second value
```

Output Block

```
x: 10
y: 20
```

Positional-Only Arguments

You can specify that a function can have ONLY positional arguments.

To specify positional-only arguments, add `/` after the arguments

```
# Function to return the positional arguments only
def my_function(name, sec,/):
    print("Hello", name, sec)

my_function("Emil", "Smith")
```


Output Block

```
Hello Emil Smith
```

Without the , / you are actually allowed to use keyword arguments even if the function expects positional arguments.

```
def my_function(name):  
    print("Hello", name)  
# defining a positional argument of name Emil  
my_function(name = "Emil")
```

Output Block

```
Hello Emil
```

```
## With , /, you will get an error if you try to use keyword arguments  
def my_function(name, /): # postional arguments expected  
    print("Hello", name)  
  
my_function(name = "Emil") # calling function and explicitly defining the keyword
```

Output Block

```
TypeError: my_function() got some positional-only arguments passed as keyword arguments:  
'name'
```

Keyword-Only Arguments

To specify that a function can have only keyword arguments, add *, before the arguments.

```
def my_function(*, name): # calling function where it expects only keyword.  
    print("Hello", name)  
# Calling the function with a keyword.  
my_function(name = "Emil")
```

Output Block

```
Hello Emil
```

Without *, you are allowed to use positional arguments even if the function expects keyword arguments.

```
def my_function(name):  
    print("Hello", name)  
# It will take this as a positional argument  
my_function("Emil")
```

Combining Positional-Only and Keyword-Only

You can combine both argument types in the same function.

Arguments before / are positional-only, and arguments after * are keyword-only.

```
def my_function(a, b, /, *, c, d):  
    return a + b + c + d # sum all parameters  
  
result = my_function(5, 10, c=15, d=20) # a and b are positional-only, c and d are  
keyword-only  
print(result) # print final result
```

Output Block

50

Python *args and **kwargs

*args and **kwargs

By default, a function must be called with the correct number of arguments.

However, sometimes you may not know how many arguments will be passed into your function.

`*args` and `**kwargs` allow functions to accept an unknown number of arguments.

Arbitrary Arguments - *args

If you do not know how many arguments will be passed into your function, add a * before the parameter name.

This way, the function will receive a tuple of arguments and can access the items accordingly

```
# Using *args to accept any number of arguments:  
  
def my_function(*kids): # * defined to take multiple arguments.  
    print("The youngest child is " + kids[2])  
# Function where multiple arguments are passed  
my_function("Emil", "Tobias", "Linus")
```

Output Block

The youngest child is Linus

What is *args?

The `*args` parameter allows a function to accept any number of positional arguments.

Inside the function, `args` becomes a tuple containing all the passed arguments.

```
# Accessing individual arguments from *args:
# Printing all the arguments and the type of the variable passed to it
def my_function(*args):
    print("Type:", type(args))
    print("First argument:", args[0])
    print("Second argument:", args[1])
    print("All arguments:", args)

my_function("Emil", "Tobias", "Linus") # Multiple values passed in function call
parameter
```

Output Block

```
Type: <class 'tuple'>
First argument: Emil
Second argument: Tobias
All arguments: ('Emil', 'Tobias', 'Linus')
```

Using *args with Regular Arguments

You can combine regular parameters with *args.

Regular parameters must come before *args.

```
def my_function(greeting, *names):
    for name in names: # iterating and printing a greeting as a return statement
        print(greeting, name)
# Function call passing a tuple
my_function("Hello", "Emil", "Tobias", "Linus") # First value is greeting, remaining 3
are names
```

Output Block

```
Hello Emil
Hello Tobias
Hello Linus
```

Practical Example with *args

*args is useful when you want to create flexible functions.

```
def my_function(*numbers): # accepts any number of arguments
    total = 0 # initialize sum
    for num in numbers: # loop through all passed values
        total += num # add each number to total
    return total # return final sum

print(my_function(1, 2, 3)) # sum of 3 numbers
print(my_function(10, 20, 30, 40)) # sum of 4 numbers
print(my_function(5)) # single value case
```

Output Block

```
6
100
5
```

```
# Finding the maximum value:

def my_function(*numbers):
    if len(numbers) == 0: # initial check to see if number is 0
        return None
    max_num = numbers[0]
    for num in numbers:
        if num > max_num:
            max_num = num
    return max_num
# Function passing multiple numbers in the form of tuples
print(my_function(3, 7, 2, 9, 1))
```

Output Block

```
9
```

Arbitrary Keyword Arguments - ****kwargs**

If you do not know how many keyword arguments will be passed into your function, add two asterisks ****** before the parameter name.

This way, the function will receive a dictionary of arguments and can access the items accordingly.

```
# Using **kwargs to accept any number of keyword arguments:

def my_function(**kid):
    print("His last name is " + kid["lname"])
# Keyword function call defined in the function call parameter
my_function(fname = "Tobias", lname = "Refsnes")
```

Output Block

```
His last name is Refsnes
```

What is ****kwargs**?

The ****kwargs** parameter allows a function to accept any number of keyword arguments.

Inside the function, **kwargs** becomes a dictionary containing all the keyword arguments.

```
def my_function(**myvar): # accepts any number of keyword arguments
    print("Type:", type(myvar)) # shows that myvar is a dictionary
    print("Name:", myvar["name"]) # access value for key 'name'
```

```
print("Age:", myvar["age"]) # access value for key 'age'
print("All data:", myvar) # print full dictionary

my_function(name="Tobias", age=30, city="Bergen") # pass keyword arguments
```

Output Block

```
Type: <class 'dict'>
Name: Tobias
Age: 30
All data: {'name': 'Tobias', 'age': 30, 'city': 'Bergen'}
```

Using **kwargs with Regular Arguments

You can combine regular parameters with **kwargs.

Regular parameters must come before **kwargs.

```
def my_function(username, **details): # username is normal arg, details collects
    extra keyword args
    print("Username:", username) # print username
    print("Additional details:") # heading for extra info

    for key, value in details.items(): # loop through keyword arguments
        print(" ", key + ":", value) # print each key-value pair

my_function("emil123", age=25, city="Oslo", hobby="coding") # call function with
extra data
```

Output Block

```
Username: emil123
Additional details:
age: 25
city: Oslo
hobby: coding
```

Combining *args and **kwargs

You can use both *args and **kwargs in the same function.

The order must be:

- 1 regular parameters
- 2 *args
- 3 **kwargs

```
def my_function(title, *args, **kwargs): # title is normal arg, args collects
    positional, kwargs collects keyword args
    print("Title:", title) # print title
```

```
print("Positional arguments:", args) # print extra positional arguments as a tuple
print("Keyword arguments:", kwargs) # print keyword arguments as a dictionary

my_function("User Info", "Emil", "Tobias", age=25, city="Oslo") # function call with
all argument types
```

Output Block

```
Title: User Info
Positional arguments: ('Emil', 'Tobias')
Keyword arguments: {'age': 25, 'city': 'Oslo'}
```

Unpacking Arguments

The `*` and `**` operators can also be used when calling functions to unpack (expand) a list or dictionary into separate arguments.

Unpacking Lists with `*`

If you have values stored in a list, you can use `*` to unpack them into individual arguments.

```
# Using * to unpack a list into arguments:
def my_function(a, b, c):
    return a + b + c

numbers = [1, 2, 3]
# passing the list to store the result in variable
result = my_function(*numbers) # Same as: my_function(1, 2, 3)
print(result)
```

Output Block

6

Unpacking Dictionaries with `**`

If you have keyword arguments stored in a dictionary, you can use `**` to unpack them.

```
def my_function(fname, lname): # function with two parameters
    print("Hello", fname, lname) # print greeting using both values

person = {"fname": "Emil", "lname": "Smith"} # dictionary with matching keys

my_function(**person) # unpack dictionary into keyword arguments
# same as: my_function(fname="Emil", lname="Smith")
```

Output Block

Hello Emil Smith

Python Variable Scope

The Concept of Scope

A variable's reach is limited to the specific area where it was defined; this boundary is what we call **scope**. If you try to access a variable outside of its designated region, Python will not recognize it.

Local Scope

Whenever a variable is initialized within a function, it is confined to that function's **local scope**.

- **Internal Access:** It can only be utilized by the code blocks located inside that specific function.
- **Isolation:** The rest of the program remains unaware of the variable's existence, ensuring that local data doesn't accidentally interfere with the wider script.

```
# A variable created inside a function is available inside that function:

def myfunc():
    x = 300 # x is the scope defined inside the function, which is local to a function
    print(x)

myfunc()
```

Output Block

300

Nested Functions

Function Within a Function

While a local variable is hidden from the main code, it remains accessible to any **nested functions** defined inside that primary function.

Inner Accessibility

If you declare a variable in an outer function, any function sitting inside it inherits the right to use that variable. This creates a chain where the inner "child" function can see the outer "parent" function's data.

```
# The local variable can be accessed from a function within the function
def myfunc():
    x = 300 # local variable inside outer function

    def myinnerfunc():
        print(x) # inner function can access outer function's variable

    myinnerfunc() # call inner function

myfunc() # call outer function
```

Output Block

300

Global Scope

A variable created in the main body of the Python code is a global variable and belongs to the global scope.

Global variables are available from within any scope, global and local.

```
# A variable created outside of a function is global and can be used by anyone:
x = 300 # Global variable defined outside the function scope
def myfunc():
    print(x) # prints the global scope
myfunc()
print(x)
```

Output Block

300
300

Naming Variables

If you operate with the same variable name inside and outside of a function, Python will treat them as two separate variables, one available in the global scope (outside the function) and one available in the local scope (inside the function).

```
# The function will print the local x, and then the code will print the global x:

x = 300 # global x variable

def myfunc():
    x = 200 # variable x local to the function
    print(x)

myfunc() # calls the function and prints 200

print(x) # prints the global value
```

Output Block

200
300

```
# if you use the global keyword, the variable belongs to the global scope.

x = 300 # global x variable

def myfunc():
    global x # Assigning the current variable x to global scope
```



```
x = 200
print(x)

myfunc() # calls the function and prints 200

print(x) # prints the global value this time it's assigned as 200
```

Output Block

```
200
200
```

Nonlocal Keyword

- The `nonlocal` keyword is used to work with variables inside nested functions.
- The `nonlocal` keyword makes the variable belong to the outer function.

```
# If you use the nonlocal keyword, the variable will belong to the outer function:

def myfunc1():
    x = "Jane" # variable in outer function

    def myfunc2():
        nonlocal x # refers to variable in enclosing function (myfunc1)
        x = "hello" # modifies outer variable

    myfunc2() # call inner function
    return x # return updated value

print(myfunc1()) # print result of function call
```

Output Block

```
hello
```

The LEGB Rule

Python follows the LEGB rule when looking up variable names, and searches for them in this order:

- Local - Inside the current function
- Enclosing - Inside enclosing functions (from inner to outer)
- Global - At the top level of the module
- Built-in - In Python's built-in namespace

```
# Understanding the LEGB rule

x = "global" # Global variable
```

```
def outer():
    x = "enclosing" # Enclosing variable inside the function
    def inner():
        x = "local" # local variable of the inner function
        print("Inner:", x)
    inner()
    print("Outer:", x)

outer()
print("Global:", x)
```

Output Block

```
Inner: local
Outer: enclosing
Global: global
```

Python Decorators

Decorator Basics

Decorators allow you to inject additional logic into an existing function without modifying its source code. They act as "wrappers" that enhance or change behavior dynamically.

The Structure

A decorator is essentially a function that accepts another function as an argument. It then returns a modified version of that function to be executed.

Implementation

To use one, define the decorator function first. You can then apply it to any target function by placing the `@decorator_name` syntax directly above the function definition.

```
# A basic decorator that uppercases the return value of the decorated function.

def changecase(func): # decorator function
    def myinner(): # wrapper function
        return func().upper() # call original function and convert result to uppercase
    return myinner # return wrapped function

@changecase # apply decorator to function
def myfunction():
    return "Hello Sally" # original function returns string

print(myfunction()) # call decorated function
```

Output Block

```
HELLO SALLY
```

- By placing `@changecase` directly above the function definition, the function `myfunction` is being "decorated" with the `changecase` function.

- The function `changepcase` is the decorator.
- The function `myfunction` is the function that gets decorated.

Multiple Decorator Calls

A decorator can be called multiple times. Just place the decorator above the function you want to decorate.

```
# Using the @changepcase decorator on two functions:

def changepcase(func): # decorator function
    def myinner(): # wrapper function
        return func().upper() # convert function output to uppercase
    return myinner # return modified function

@changepcase # apply decorator to myfunction
def myfunction():
    return "Hello Sally" # original string

@changepcase # apply decorator to otherfunction
def otherfunction():
    return "I am speed!" # original string

print(myfunction())
print(otherfunction())
```

Output Block

```
HELLO SALLY
I AM SPEED!
```

Arguments in the Decorated Function

Functions that require arguments can also be decorated, just make sure you pass the arguments to the wrapper function.

```
# Functions with arguments can also be decorated:

def changepcase(func):
    def myinner(x):
        return func(x).upper() # changes the function and the argument to Uppercase
    return myinner

@changepcase
def myfunction(nam):
    return "Hello " + nam

print(myfunction("John")) # passing the name as John
```

Output Block

```
HELLO JOHN
```

*args and **kwargs

Sometimes the decorator function has no control over the arguments passed from decorated function, to solve this problem, add `(*args, **kwargs)` to the wrapper function, this way the wrapper function can accept any number, and any type of arguments, and pass them to the decorated function.

```
# Secure the function with *args and **kwargs arguments:

def changecase(func): # decorator function
    def myinner(*args, **kwargs): # wrapper that accepts any arguments
        return func(*args, **kwargs).upper() # call original function and convert result
        to uppercase
    return myinner # return wrapped function

@changecase # apply decorator
def myfunction(*nam):
    return "Hello " + nam[1] # return greeting string

print(myfunction("John", "Mary"))
```

Output Block

```
HELLO MARY
```

Decorators With Arguments

Passing Parameters

To create a decorator that accepts its own arguments, you must introduce an additional **wrapper level**. This essentially involves nesting a second function inside the decorator to handle the extra input.

The Extra Layer

The outermost function captures the decorator's arguments, the middle function captures the target function, and the innermost function executes the actual logic. This structure allows you to customize the decorator's behavior for different scenarios.

```
# A decorator factory that takes an argument and transforms the casing based on the
argument value.

def changecase(n): # outer function that takes a parameter
    def changecase(func): # decorator function
        def myinner(): # wrapper function
            if n == 1: # check condition from outer function
                a = func().lower() # convert result to lowercase
            else:
                a = func().upper() # convert result to uppercase
            return a # return modified result
        return myinner # return wrapper
    return changecase # return decorator

@changecase(1) # apply decorator with argument
def myfunction():
    return "Hello Linus" # original function
```

```
print(myfunction())
```

Output Block

```
hello linux
```

Multiple Decorators

- You can use multiple decorators on one function.
- This is done by placing the decorator calls on top of each other.
- Decorators are called in the reverse order, starting with the one closest to the function.

```
# One decorator for upper case, and one for adding a greeting

def changecase(func): #Decorator to change the case of the function
    def myinner():
        return func().upper()
    return myinner

def addgreeting(func): # Decorator to add the greeting before the function is called
    def myinner():
        return "Hello " + func() + " Have a good day!"
    return myinner

@changecase
@addgreeting
def myfunction():
    return "Tobias"

print(myfunction())
```

Output Block

```
HELLO TOBIAS HAVE A GOOD DAY!
```

Handling Arguments in Decorated Functions

You can apply decorators to functions that take inputs; you simply need to ensure those arguments are passed into the **wrapper function**. By defining the inner wrapper to accept parameters, it can successfully relay them to the original function during execution.

```
# Functions with arguments can also be decorated
def changecase(func):
    # Inner function to wrap the original logic and handle arguments
    def myinner(x):
        # Calls the original function and converts the result to uppercase
        return func(x).upper()
    return myinner
```

```
# Applying the decorator to modify the output of myfunction
@changepcase
def myfunction(nam):
    return "Hello " + nam

# Executing the decorated function
print(myfunction("John"))
```

Output Block

```
HELLO JOHN
```

Flexible Argument Handling with `\*args` and `\*\*kwargs`

When a decorator needs to support functions with unpredictable signatures, you should use `*args` and `**kwargs` within the wrapper. This configuration allows the wrapper function to capture any number of positional and keyword arguments, ensuring they are seamlessly forwarded to the decorated function regardless of its specific requirements.

```
# Secure the function with *args and **kwargs arguments

def changepcase(func):
    # Inner function uses *args and **kwargs to accept any number of arguments
    def myinner(*args, **kwargs):
        # Passes arguments through to the original function and uppercases the result
        return func(*args, **kwargs).upper()
    return myinner

# Decorator applied to a function with one argument
@changepcase
def myfunction(nam):
    return "Hello " + nam

# Prints the modified, uppercase string
print(myfunction("John"))
```

Output Block

```
HELLO JOHN
```

Decorator With Arguments

Decorators can accept their own arguments by adding another wrapper level.

```
def changepcase(n):
    # The outer function acts as a factory to pass arguments to the decorator
    def decorator(func):
        # The actual decorator that wraps the target function
        def myinner():
            # Conditional logic based on the argument 'n' passed to the decorator
```

```

        if n == 1:
            a = func().lower()
        else:
            a = func().upper()
        return a
    return myinner
return decorator

```

```

# Applying the decorator with an argument to specify lowercase conversion
@changepcase(1)
def myfunction():
    return "Hello Linus"

# Outputs the result based on the logic defined in the nested decorator
print(myfunction())

```

Output Block

```
hello linus
```

Multiple Decorators

- **Layered Application:** A single function can be wrapped by multiple decorators simultaneously.
- **Implementation:** To apply several decorators, stack the decorator calls directly on top of each other above the function definition.
- **Execution Order:** Decorators trigger in **reverse order** (bottom-to-top), meaning the one positioned closest to the function executes first.

```

# One decorator for upper case, and one for adding a greeting

def changepcase(func):
    # Wrapper function to uppercase the result
    def myinner():
        return func().upper()
    return myinner

def addgreeting(func):
    # Wrapper function to wrap the string with a greeting
    def myinner():
        return "Hello " + func() + " Have a good day!"
    return myinner

# Decorators are applied from the bottom up (closest to the function first)
@changepcase
@addgreeting
def myfunction():
    return "Tobias"

# Executes addgreeting first, then changepcase on the resulting string
print(myfunction())

```

Output Block

```
HELLO TOBIAS HAVE A GOOD DAY!
```

Preserving Function Metadata

Functions in Python has metadata that can be accessed using the `__name__` and `__doc__` attributes.

```
def myfunction():
    return "Have a great day!"

# Accessing the __name__ attribute to print the function's original name
print(myfunction.__name__)
```

Output Block

```
myfunction
```

```
# Try returning the name from a decorated function and you will not get the same
result

def changecase(func):
    # The wrapper function defined inside the decorator
    def myinner():
        return func().upper()
    return myinner

# The decorator replaces 'myfunction' with the 'myinner' closure
@changecase
def myfunction():
    return "Have a great day!"

# This will print 'myinner' instead of 'myfunction'
# because the original function metadata is shadowed by the wrapper
print(myfunction.__name__)
```

Output Block

```
myinner
```

To fix this, Python has a built-in function called `functools.wraps` that can be used to preserve the original function's name and docstring.

```
# Import functools.wraps to preserve the original function name and docstring.
import functools

def changecase(func):
    # Use @functools.wraps to copy the original function's metadata to the wrapper
    @functools.wraps(func)
    def myinner():
        # Transformation logic
        return func().upper()
```



```
        return myinner

# Applying the decorator
@changepcase
def myfunction():
    return "Have a great day!"

# Now this correctly prints 'myfunction' instead of 'myinner'
print(myfunction.__name__)
```

Output Block

```
myfunction
```

Python Lambda Functions

- **Definition:** A lambda function is a compact, anonymous function that is defined without a specific name.
- **Capacity and Constraints:** It can accept an unlimited number of arguments, but it is restricted to a **single expression**.
- **Syntax Structure:** The standard format is written as `lambda arguments : expression`.
- **Execution Logic:** When the function is called, the expression is evaluated and the resulting value is automatically returned.

```
# Add 10 to argument a, and return the result
x = lambda a : a + 10
print(x(5))
```

Output Block

```
15
```

```
# Lambda functions can take any number of arguments.

# Multiply argument a with argument b and return the result:
x = lambda a, b : a * b
print(x(5, 6))
```

Output Block

```
30
```

```
# Summarize argument a, b, and c and return the result.

x = lambda a, b, c : a + b + c
print(x(5, 6, 2))
```

Why Use Lambda Functions?

The power of lambda is better shown when you use them as an anonymous function inside another function.

Say you have a function definition that takes one argument, and that argument will be multiplied with an unknown number.

```
def myfunc(n):  
    # Returns a lambda function that multiplies its input 'a' by the outer variable 'n'  
    return lambda a : a * n
```

```
# Use that function definition to make a function that always doubles the number you  
send in.
```

```
def myfunc(n):  
    # Returns a closure in the form of a lambda function  
    return lambda a : a * n  
  
# Create a specific instance of the function that multiplies input by 2  
mydoubler = myfunc(2)  
  
# Pass 11 into the newly created doubler function  
print(mydoubler(11))
```

```
# Or, use the same function definition to make a function that always triples the  
number you send in.
```

```
def myfunc(n):  
    # Returns a lambda function acting as a multiplier  
    return lambda a : a * n  
  
# Create a specific function instance that multiplies input by 3  
mytripler = myfunc(3)  
  
# Executes the tripler with 11 as the argument  
print(mytripler(11))
```

Or, use the same function definition to make both functions, in the same program.

```
def myfunc(n):  
    # Higher-order function returning a lambda for multiplication  
    return lambda a : a * n  
  
# Instantiate a doubler (n=2) and a tripler (n=3) from the same factory  
mydoubler = myfunc(2)  
mytrippler = myfunc(3)  
  
# Test both instances with the same input value  
print(mydoubler(11))  
print(mytrippler(11))
```

Output Block

```
22  
33
```

Lambda with Built-in Functions

Common Use Cases: Lambda functions are frequently utilized alongside built-in Python functions such as `map()`, `filter()`, and `sorted()`.

Using Lambda with `map()`: The `map()` function is designed to apply a specific function to every individual item within an iterable.

```
# Double all numbers in a list.  
numbers = [1, 2, 3, 4, 5]  
doubled = list(map(lambda x: x * 2, numbers))  
print(doubled)
```

Output Block

```
[2, 4, 6, 8, 10]
```

Using Lambda with `filter()`

The `filter()` function creates a list of items for which a function returns `True`.

```
# Filter out odd numbers from a list.  
  
numbers = [1, 2, 3, 4, 5, 6, 7, 8]  
odd_numbers = list(filter(lambda x: x % 2 != 0, numbers))  
print(odd_numbers)
```

Output Block

```
[1, 3, 5, 7]
```

Using Lambda with sorted()

The `sorted()` function can use a lambda as a key for custom sorting.

```
# Sort a list of tuples by the second element:
students = [("Emil", 25), ("Tobias", 22), ("Linus", 28)]

# Uses a lambda function as the sorting key to target index 1 (the age)
sorted_students = sorted(students, key=lambda x: x[1])

# Displays the list reordered by age in ascending order
print(sorted_students)
```

Output Block

```
[('Tobias', 22), ('Emil', 25), ('Linus', 28)]
```

```
# Sort strings by length:
words = ["apple", "pie", "banana", "cherry"]

# Uses the len() function within a lambda to determine the sort order
sorted_words = sorted(words, key=lambda x: len(x))

# Displays the list ordered from shortest to longest string
print(sorted_words)
```

Output Block

```
['pie', 'apple', 'banana', 'cherry']
```

Recursion

Recursion is when a function calls itself.

Recursion is a common mathematical and programming concept. It means that a function calls itself. This has the benefit of meaning that you can loop through data to reach a result.

The developer should be very careful with recursion as it can be quite easy to slip into writing a function which never terminates, or one that uses excess amounts of memory or processor power. However, when written correctly recursion can be a very efficient and mathematically-elegant approach to programming.

```
# A simple recursive function that counts down from 5.

def countdown(n):
    if n <= 0:
        print("Done!")
    else:
        print(n)
        countdown(n - 1)
```

```
countdown(5)
```

Output Block

```
5
4
3
2
1
Done!
```

Base Case and Recursive Case

- **The Base Case:** This is the essential condition that terminates the recursive process and prevents infinite loops.
- **The Recursive Case:** This component involves the function invoking itself, typically using a modified argument to move closer toward the base case.
- **Error Prevention:** In the absence of a base case, the function would execute indefinitely, ultimately resulting in a **stack overflow error**.

```
# Identifying base case and recursive case.

def factorial(n):
    # Base case: provides a stopping condition to prevent infinite recursion
    if n == 0 or n == 1:
        return 1
    # Recursive case: the function calls itself with a reduced version of the problem
    else:
        return n * factorial(n - 1)

# Calculates 5 * 4 * 3 * 2 * 1
print(factorial(5))
```

Output Block

```
120
```

Fibonacci Sequence

The Fibonacci sequence is a classic example where each number is the sum of the two preceding ones. The sequence starts with 0 and 1:

0, 1, 1, 2, 3, 5, 8, 13, ...

The sequence continues indefinitely, with each number being the sum of the two preceding ones.

We can use recursion to find a specific number in the sequence.

```
# Find the 7th number in the Fibonacci sequence.

def fibonacci(n):
    # Base case: returns n if it is 0 or 1
    if n <= 1:
        return n
    # Recursive case: sums the two preceding numbers in the sequence
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

# Executes the function to find the 7th Fibonacci number (13)
print(fibonacci(7))
```

Output Block

13

Recursion with Lists

- **Sequential Processing:** Recursion provides a method for navigating lists by operating on a single element during each function call.
- **Divide and Conquer:** Typically, the function processes the first item (the head) and then recursively calls itself with the remainder of the list (the tail) until the list is empty.

```
# Calculate the sum of all elements in a list.

def sum_list(numbers):
    # Base case: if the list is empty, the sum is 0
    if len(numbers) == 0:
        return 0
    # Recursive case: add the first element to the sum of the remaining list
    else:
        return numbers[0] + sum_list(numbers[1:])

my_list = [1, 2, 3, 4, 5]
# Prints the total sum calculated through recursive slicing
print(sum_list(my_list))
```

Output Block

15

```
# Find the maximum value in a list:

def find_max(numbers):
    # Base case: if the list has only one element, that element is the maximum
    if len(numbers) == 1:
        return numbers[0]
    else:
        # Recursive case: find the maximum of the tail of the list
        max_of_rest = find_max(numbers[1:])
        # Compare the first element with the maximum found in the rest of the list
```

```
        return numbers[0] if numbers[0] > max_of_rest else max_of_rest

my_list = [3, 7, 2, 9, 1]
# Prints the highest number found through recursive comparisons
print(find_max(my_list))
```

Output Block

9

Recursion Depth Limit

- **Safety Constraint:** Python imposes a specific restriction on the maximum depth of the call stack to prevent infinite loops from crashing the system.
- **Standard Threshold:** The default recursion limit is typically set to approximately **1000** consecutive calls.
- **System Protection:** Once this boundary is reached, Python raises a `RecursionError` to protect memory resources.

```
# Check the recursion limit:
import sys
print(sys.getrecursionlimit())
```

Output Block

3000

```
import sys

# Increases the maximum depth of the Python interpreter stack
sys.setrecursionlimit(2000)

# Verifies and prints the newly set recursion limit
print(sys.getrecursionlimit())
```

Python Generators

Generators are functions that can pause and resume their execution.

When a generator function is called, it returns a generator object, which is an iterator.

The code inside the function is not executed yet, it is only compiled. The function only executes when you iterate over the generator.

```
# A simple generator function:

def my_generator():
```

```
# yield suspends function execution and sends a value back to the caller
yield 1
yield 2
yield 3

# Iterating over the generator object produced by the function call
for value in my_generator():
    # Prints each yielded value until the generator is exhausted
    print(value)
```

Output Block

```
1
2
3
```

Generators allow you to iterate over data without storing the entire dataset in memory. Instead of using `return`, generators use the `yield` keyword.

The yield Keyword

The `yield` keyword is what makes a function a generator.

When `yield` is encountered, the function's state is saved, and the value is returned. The next time the generator is called, it continues from where it left off.

```
# Generator that yields numbers

def count_up_to(n):
    count = 1
    while count <= n:
        # yield pauses the function and returns the current count
        yield count
        # When resumed, the function continues from this point
        count += 1

# Creating a generator object and iterating through it
for num in count_up_to(5):
    # Prints numbers 1 through 5
    print(num)
```

Output Block

```
1
2
3
4
5
```

Unlike `return`, which terminates the function, `yield` pauses it and can be called multiple times.

Generators Saves Memory

Generators are memory-efficient because they generate values on-the-fly instead of storing everything in memory.

For large datasets, generators save memory.

```
# Generator for large sequences:
def large_sequence(n):
    for i in range(n):
        yield i

# This doesn't create a million numbers in memory
gen = large_sequence(1000000)

# Fetch first value
print(next(gen))
# Fetch second value
print(next(gen))
# Fetch third value
print(next(gen))
```

Output Block

```
0
1
2
```

Using next() with Generators

You can manually iterate through a generator using the `next()` function

```
def simple_gen():
    # Yields individual strings one at a time
    yield "Emil"
    yield "Tobias"
    yield "Linus"

gen = simple_gen()

# Fetch first yield
print(next(gen))
# Fetch second yield
print(next(gen))
# Fetch third yield
print(next(gen))
```

Output Block

```
Emil
Tobias
Linus
```

Generator Expressions

- **Syntactic Similarity:** Generator expressions share a syntax nearly identical to list comprehensions.

- **Notation Difference:** To define a generator instead of a list, simply use **parentheses** `()` in place of square brackets `[]`.
- **Memory Efficiency:** Unlike comprehensions that build the entire collection in memory, these expressions create a generator object that yields items lazily (one at a time).

```
# List comprehension vs generator expression.

# List comprehension - creates a list
list_comp = [x * x for x in range(5)]
print(list_comp)

# Generator expression - creates a generator
gen_exp = (x * x for x in range(5))
print(gen_exp)
print(list(gen_exp))
```

Output Block

```
[0, 1, 4, 9, 16]
<generator object <genexpr> at 0x00000021681435A40>
[0, 1, 4, 9, 16]
```

```
# Using a generator expression with sum

# Calculate sum of squares without creating a list
total = sum(x * x for x in range(10))
print(total)
```

Output Block

```
285
```

Fibonacci Sequence Generator

Generators can be used to create the Fibonacci sequence.

It can continue generating values indefinitely, without running out of memory.

```
# Generate 100 Fibonacci numbers.

def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

# Get first 100 Fibonacci numbers
gen = fibonacci()
for _ in range(100):
    print(next(gen))
```

Generator Methods

Generators have special methods for advanced control.

send() Method

The send() method allows you to send a value to the generator.

```
def echo_generator():
    # Infinite loop to keep the generator alive
    while True:
        # yield pauses execution and can receive values sent via .send()
        received = yield
        print("Received:", received)

gen = echo_generator()

# Prime the generator to reach the first yield statement
next(gen)

# Send data into the generator
gen.send("Hello")
# Send more data
gen.send("World")
```

Output Block

```
Received: Hello
Received: World
```

close() Method

The `close()` method stops the generator.

```
def my_gen():
    try:
        yield 1
        yield 2
        yield 3
    finally:
        # This block executes when the generator is closed or exhausted
        print("Generator closed")

gen = my_gen()

# Fetch the first value
print(next(gen))

# Manually terminate the generator
gen.close()
```

Output Block

1
Generator closed

Python Library - NumPy

Starting with a basic introduction and ends up with creating and plotting random data sets, and working with NumPy functions.

Basic	Random	ufunc
Introduction	Random Intro	ufunc Intro
Getting Started	Data Distribution	Create Function
Creating Arrays	Random Permutation	Simple Arithmetic
Array Indexing	Seaborn Module	Rounding Decimals
Array Slicing	Normal Dist.	Logs
Data Types	Binomial Dist.	Summations
Copy vs View	Poisson Dist.	Products
Array Shape	Uniform Dist.	Differences
Array Reshape	Logistic Dist.	Finding LCM
Array Iterating	Multinomial Dist.	Finding GCD
Array Join	Exponential Dis.	Trigonometric
Array Split	Chi Square Dist.	Hyperbolic
Array Search	Rayleigh Dist.	Set Operations
Array Sort	Pareto Dist.	
Array Filter	Zipf Dist.	

```
import numpy as np

#create an array using numpy
arr = np.array([1,2,3,4,5])
print(arr) # prints the array in console
print(type(arr)) # returns the type of the array
```

Output Block

```
[1 2 3 4 5]
<class 'numpy.ndarray'>
```

What is NumPy?

NumPy is a Python library used for working with arrays.

It also has functions for working in domain of linear algebra, fourier transform, and matrices.

- NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely.

- NumPy stands for Numerical Python.

```
import numpy as np
# what if we wanted to do a mathematical operation on the below two lists? Is this the
correct answer?
x = [1,2,3]
y= [2,3,4]
x+y # just concatenates the list items
```

Output Block

```
[1, 2, 3, 2, 3, 4]
```

```
# The answer is no. That's where numpy can help. It's core to numerical computing and
other packages such as Pandas
x_arr = np.array(x)
y_arr = np.array(y)
x_arr+ y_arr # actually performs the sum of two arrays
```

Output Block

```
array([3, 5, 7])
```

```
# let's use some other functions in numpy array
# Creates an array of 20 integers from 0 to 19
print(np.arange(20))
# Creates an array of integer numbers from 5 to 101 with 5 as an interval
print(np.arange(5,101,5))
```

Output Block

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19]
[ 5 10 15 20 25 30 35 40 45 50 55 60 65 70 75 80 85 90
95 100]
```

```
# divides the interval between 1 and 100 in 20 equal parts
# Generates 20 evenly spaced points including both 1 and 100
print(np.linspace(1, 100, 20))
print('\n')
print(np.round(np.linspace(1,100,20),2)) # with 2 decimal places rounding
```

Output Block

```
[ 1.  6.21052632 11.42105263 16.63157895 21.84210526
27.05263158 32.26315789 37.47368421 42.68421053 47.89473684
53.10526316 58.31578947 63.52631579 68.73684211 73.94736842
79.15789474 84.36842105 89.57894737 94.78947368 100. ]
```

```
[ 1.  6.21 11.42 16.63 21.84 27.05 32.26 37.47 42.68 47.89
53.11 58.32 63.53 68.74 73.95 79.16 84.37 89.58 94.79 100. ]
```

```
# Create n dimensional arrays
x = np.arange(20).reshape(5,4)
print(x)

# Create a multi dimensional array
y = np.arange(20).reshape(2,5,2) # 2 x 5 x 2 array
print(y)
```

Output Block

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]
 [16 17 18 19]]

[[[ 0  1]
 [ 2  3]
 [ 4  5]
 [ 6  7]
 [ 8  9]]

 [[10 11]
 [12 13]
 [14 15]
 [16 17]
 [18 19]]]
```

```
# explore features of the arrays
# Returns a tuple representing array dimensions (rows, columns)
print(x.shape)

# Identifies the object type (typically <class 'numpy.ndarray'>)
print(type(x))

# Returns the number of dimensions (axes) of the array
print(x.ndim)

# Returns the total count of elements across all dimensions
print(x.size)

# Describes the data type of the elements (e.g., int64, float32)
print(x.dtype)
```

Output Block

```
(5, 4)
<class 'numpy.ndarray'>
2
20
int64
```

```
# Other arrays
x = np.zeros(15).reshape(3,5) # all zeros
print(x)

y = np.ones(15).reshape(3,5) # all ones
print(y)
```

Output Block

```
[[0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]]
[[1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]]
```

```
# Random numbers from uniform distribution between 0 and 1
a = np.random.random(15).reshape(3,5)
print(a)
print('\n')

# Random numbers from standard normal distribution mean= 0 and sd = 1
b = np.random.randn(15).reshape(3,5)
print(b)
```

Output Block

```
[[0.78824134 0.74331649 0.53979764 0.60365034 0.70198569]
 [0.74968811 0.57951656 0.96428713 0.96272354 0.48473611]
 [0.07658163 0.18710392 0.95555738 0.26950211 0.31786098]]

[[-0.70857891 0.38721294 -0.50705635 -0.08295546 -0.3777288 ]
 [ 1.84404165 0.09181126 0.56530094 1.34511658 -0.43502033]
 [-0.27147523 0.08008755 -0.10354789 1.44749537 -0.17824081]]
```

Pandas: Overview and Applications

Pandas is a powerful, open-source Python library specifically designed for **data manipulation and analysis**. It provides high-performance data structures that make working with "relational" or "labeled" data both easy and intuitive.

- **Core Data Structures:**

Series: A 1D labeled array capable of holding any data type.

- **DataFrame:** A 2D labeled data structure (like a table or spreadsheet) with columns of potentially different types.

Common Uses:

- **Data Cleaning:** Handling missing data, filtering rows, and renaming columns.
- **Data Transformation:** Merging, joining, and reshaping datasets.
- **Statistical Analysis:** Calculating mean, median, correlations, and descriptive statistics.

Real-World Applications:

- **Finance:** Analyzing stock market trends and risk management.
- **Data Science:** Pre-processing raw data for machine learning models.
- **Economics:** Managing large-scale longitudinal datasets and social surveys.

Seaborn: Statistical Data Visualization

Seaborn is a Python data visualization library based on **Matplotlib** that provides a high-level interface for drawing attractive and informative statistical graphics.

- **Key Features:** It simplifies complex visualizations like heatmaps, time series, and violin plots with minimal code.
- **Integration:** It is closely integrated with **Pandas** data structures, allowing you to pass DataFrames directly into plotting functions.
- **Aesthetics:** Seaborn automatically handles sophisticated styling, color palettes, and statistical overlays (like regression lines) to make data more readable and professional.

```
import pandas as pd
import seaborn as sns
```

Loading the Titanic dataset.

Loading the titanic dataset from Seaborn and displays the first five rows.

```
titanic = sns.load_dataset('titanic') # Load the Titanic dataset from Seaborn
titanic.head() # Display the first five rows of the dataset
```

Output Block


```
survived pclass sex age sibsp parch fare embarked class \
0 0 3 male 22.0 1 0 7.2500 S Third
1 1 1 female 38.0 1 0 71.2833 C First
2 1 3 female 26.0 0 0 7.9250 S Third
3 1 1 female 35.0 1 0 53.1000 S First
4 0 3 male 35.0 0 0 8.0500 S Third
```

```
who adult_male deck embark_town alive alone
0 man True NaN Southampton no False
1 woman False C Cherbourg yes False
2 woman False NaN Southampton yes True
3 woman False C Southampton yes False
4 man True NaN Southampton no True
```

Export Titanic Dataset to CSV

```
titanic_csv = titanic.to_csv('titanic_seaborn.csv', index=False) # Export the dataset
to a CSV file without row indices
print("CSV Exported as titanic_seaborn.csv") # Confirm that the file has been saved
```

Output Block

```
CSV Exported as titanic_seaborn.csv
```

Read the Data from the Exported CSV File.

```
df = pd.read_csv('titanic_seaborn.csv')
print(type(df)) # Returns the datatype
print(df.shape) # Returns the (rows, columns)
df.head(20) # Returns the first 5 rows
```

Output Block

```

<class 'pandas.core.frame.DataFrame'>
(891, 15)

survived pclass sex age sibsp parch fare embarked class \
0 0 3 male 22.0 1 0 7.2500 S Third
1 1 1 female 38.0 1 0 71.2833 C First
2 1 3 female 26.0 0 0 7.9250 S Third
3 1 1 female 35.0 1 0 53.1000 S First
4 0 3 male 35.0 0 0 8.0500 S Third
5 0 3 male NaN 0 0 8.4583 Q Third
6 0 1 male 54.0 0 0 51.8625 S First
7 0 3 male 2.0 3 1 21.0750 S Third
8 1 3 female 27.0 0 2 11.1333 S Third
9 1 2 female 14.0 1 0 30.0708 C Second
10 1 3 female 4.0 1 1 16.7000 S Third
11 1 1 female 58.0 0 0 26.5500 S First
12 0 3 male 20.0 0 0 8.0500 S Third
13 0 3 male 39.0 1 5 31.2750 S Third
14 0 3 female 14.0 0 0 7.8542 S Third
15 1 2 female 55.0 0 0 16.0000 S Second
16 0 3 male 2.0 4 1 29.1250 Q Third
17 1 2 male NaN 0 0 13.0000 S Second
18 0 3 female 31.0 1 0 18.0000 S Third
19 1 3 female NaN 0 0 7.2250 C Third

who adult_male deck embark_town alive alone
0 man True NaN Southampton no False
1 woman False C Cherbourg yes False
2 woman False NaN Southampton yes True
3 woman False C Southampton yes False
4 man True NaN Southampton no True
5 man True NaN Queenstown no True
6 man True E Southampton no True
7 child False NaN Southampton no False
8 woman False NaN Southampton yes False
9 child False NaN Cherbourg yes False
10 child False G Southampton yes False
11 woman False C Southampton yes True
12 man True NaN Southampton no True
13 man True NaN Southampton no False
14 child False NaN Southampton no True
15 woman False NaN Southampton yes True
16 child False NaN Queenstown no False
17 man True NaN Southampton yes True
18 woman False NaN Southampton no False
19 woman False NaN Cherbourg yes True

```

```
df.sample(5) # Returns the random rows from the dataset
```

Output Block

```
survived pclass sex age sibsp parch fare embarked class \
810 0 3 male 26.0 0 0 7.8875 S Third
799 0 3 female 30.0 1 1 24.1500 S Third
231 0 3 male 29.0 0 0 7.7750 S Third
70 0 2 male 32.0 0 0 10.5000 S Second
215 1 1 female 31.0 1 0 113.2750 C First
```

```
who adult_male deck embark_town alive alone
810 man True NaN Southampton no True
799 woman False NaN Southampton no False
231 man True NaN Southampton no True
70 man True NaN Southampton no True
215 woman False D Cherbourg yes False
```

```
df.info() # Gives the outline of the dataset, number of entries and column name and
types.
```

Output Block

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
# Column Non-Null Count Dtype
---  ---
0 survived 891 non-null int64
1 pclass 891 non-null int64
2 sex 891 non-null object
3 age 714 non-null float64
4 sibsp 891 non-null int64
5 parch 891 non-null int64
6 fare 891 non-null float64
7 embarked 889 non-null object
8 class 891 non-null object
9 who 891 non-null object
10 adult_male 891 non-null bool
11 deck 203 non-null object
12 embark_town 889 non-null object
13 alive 891 non-null object
14 alone 891 non-null bool
dtypes: bool(2), float64(2), int64(4), object(7)
memory usage: 92.4+ KB
```

```
df.tail() # Returns the last 5 rows
```

Output Block

```

survived pclass sex age sibsp parch fare embarked class \
886 0 2 male 27.0 0 0 13.00 S Second
887 1 1 female 19.0 0 0 30.00 S First
888 0 3 female NaN 1 2 23.45 S Third
889 1 1 male 26.0 0 0 30.00 C First
890 0 3 male 32.0 0 0 7.75 Q Third

```

```

who adult_male deck embark_town alive alone
886 man True NaN Southampton no True
887 woman False B Southampton yes True
888 woman False NaN Southampton no False
889 man True C Cherbourg yes True
890 man True NaN Queenstown no True

```

```
df.describe() # Generates the descriptive stats only the numerical values
```

Output Block

```

survived pclass age sibsp parch fare
count 891.000000 891.000000 714.000000 891.000000 891.000000 891.000000
mean 0.383838 2.308642 29.699118 0.523008 0.381594 32.204208
std 0.486592 0.836071 14.526497 1.102743 0.806057 49.693429
min 0.000000 1.000000 0.420000 0.000000 0.000000 0.000000
25% 0.000000 2.000000 20.125000 0.000000 0.000000 7.910400
50% 0.000000 3.000000 28.000000 0.000000 0.000000 14.454200
75% 1.000000 3.000000 38.000000 1.000000 0.000000 31.000000
max 1.000000 3.000000 80.000000 8.000000 6.000000 512.329200

```

Check for Missing Values

Use `isnull().sum()` to check the missing values in each column.

```
df.isnull().sum() # To check the columns and returns the number of null values in each column
```

Output Block

```
survived 0
pclass 0
sex 0
age 177
sibsp 0
parch 0
fare 0
embarked 2
class 0
who 0
adult_male 0
deck 688
embark_town 2
alive 0
alone 0
dtype: int64
```

Value counts for the 'sex' column

```
df['sex'].value_counts() # Gets the gender counts
df['embark_town'].value_counts() # Returns the boarded point of the people
```

Output Block

```
embark_town
Southampton 644
Cherbourg 168
Queenstown 77
Name: count, dtype: int64
```

Group by 'class' and Calculate the Mean Age:

Group the DataFrame by 'class' and calculate the mean age of each class.

```
df.groupby('class')['age'].mean()
```

Output Block

```
class
First 38.233441
Second 29.877630
Third 25.140620
Name: age, dtype: float64
```

```
df.groupby('sex')['survived'].mean()
```

Output Block

```
sex
female 0.742038
male 0.188908
Name: survived, dtype: float64
```

Sort by Fare Descending

Sort the Dataframe by 'Fare' column in descending

```
df.sort_values('fare', ascending=False).head()
```

Output Block

```
survived pclass sex age sibsp parch fare embarked class \
679 1 1 male 36.0 0 1 512.3292 C First
258 1 1 female 35.0 0 0 512.3292 C First
737 1 1 male 35.0 0 0 512.3292 C First
88 1 1 female 23.0 3 2 263.0000 S First
438 0 1 male 64.0 1 4 263.0000 S First
```

```
who adult_male deck embark_town alive alone
679 man True B Cherbourg yes False
258 woman False NaN Cherbourg yes True
737 man True B Cherbourg yes True
88 woman False C Southampton yes False
438 man True C Southampton no False
```

Filter the data to return the people with Age > 30.

```
df[df['age'] > 30].head()
```

Output Block

```
survived pclass sex age sibsp parch fare embarked class \
1 1 1 female 38.0 1 0 71.2833 C First
3 1 1 female 35.0 1 0 53.1000 S First
4 0 3 male 35.0 0 0 8.0500 S Third
6 0 1 male 54.0 0 0 51.8625 S First
11 1 1 female 58.0 0 0 26.5500 S First
```

```
who adult_male deck embark_town alive alone
1 woman False C Cherbourg yes False
3 woman False C Southampton yes False
4 man True NaN Southampton no True
6 man True E Southampton no True
11 woman False C Southampton yes True
```

```
df['family_size'] = df['sibsp'] + df['parch'] + 1
df[['sibsp', 'parch', 'family_size']].head()
```

Output Block

```
sibsp parch family_size
0 1 0 2
1 1 0 2
2 0 0 1
3 1 0 2
4 0 0 1
```

Drop Rows with Missing Age

Drop rows where the 'age' is NULL using the `dropna` function.

```
df_no_missing_age = df.dropna(subset=['age'])
df_no_missing_age.shape
```

Output Block

```
(714, 16)
```

```
np.round(pd.pivot_table(df, values='survived', index='sex', columns='class',
aggfunc='mean')*100,1)
```

Output Block

```
class First Second Third
sex
female 96.8 92.1 50.0
male 36.9 15.7 13.5
```

Select Specific Columns

Select the columns 'age', 'fare' and 'survived'.

```
df[['age', 'fare', 'survived']].head() # returns the first 5 rows of specified
columns.
```

Output Block

```
age fare survived
0 22.0 7.2500 0
1 38.0 71.2833 1
2 26.0 7.9250 1
3 35.0 53.1000 1
4 35.0 8.0500 0
```

Set 'class' Column as Index

```
df_class_index = df.set_index('class') # Set the 'class' column as the DataFrame index
df_class_index.head() # Display the first five rows with the new index
```

Output Block

```
survived pclass sex age sibsp parch fare embarked who \
class
Third 0 3 male 22.0 1 0 7.2500 S man
First 1 1 female 38.0 1 0 71.2833 C woman
Third 1 3 female 26.0 0 0 7.9250 S woman
First 1 1 female 35.0 1 0 53.1000 S woman
Third 0 3 male 35.0 0 0 8.0500 S man

adult_male deck embark_town alive alone family_size
class
Third True NaN Southampton no False 2
First False C Cherbourg yes False 2
Third False NaN Southampton yes True 1
First False C Southampton yes False 2
Third True NaN Southampton no True 1
```

Reset the Index

```
## Reset Index

df_reset = df_class_index.reset_index()
df_reset.head()
```

Output Block

```
class survived pclass sex age sibsp parch fare embarked \
0 Third 0 3 male 22.0 1 0 7.2500 S
1 First 1 1 female 38.0 1 0 71.2833 C
2 Third 1 3 female 26.0 0 0 7.9250 S
3 First 1 1 female 35.0 1 0 53.1000 S
4 Third 0 3 male 35.0 0 0 8.0500 S

who adult_male deck embark_town alive alone family_size
0 man True NaN Southampton no False 2
1 woman False C Cherbourg yes False 2
2 woman False NaN Southampton yes True 1
3 woman False C Southampton yes False 2
4 man True NaN Southampton no True 1
```

Pandas: loc vs. iloc

- **loc (Label-based Indexing):** Used to access data by the **name** of the rows and columns. It includes the last element in a slice (e.g., `0:2` returns rows 0, 1, and 2).

- **iloc (Integer-based Indexing):** Used to access data by its **numerical position** (index starting at 0). Like standard Python slicing, it excludes the last element in a slice (e.g., `0:2` returns positions 0 and 1).

```
# Using loc and iloc
# loc
print(df.loc[0:5, ['age', 'fare']])
```

Output Block

```
age fare
0 22.0 7.2500
1 38.0 71.2833
2 26.0 7.9250
3 35.0 53.1000
4 35.0 8.0500
5 NaN 8.4583
```

```
# iloc
print(df.iloc[2:3, :])
```

Output Block

```
survived pclass sex age sibsp parch fare embarked class who \
2 1 3 female 26.0 0 0 7.925 S Third woman

adult_male deck embark_town alive alone family_size
2 False NaN Southampton yes True 1
```

```
df.columns # returns the column names
```

Output Block

```
Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
      'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',
      'alive', 'alone'],
      dtype='object')
```

```
df.index # returns the index names
```

Output Block

```
RangeIndex(start=0, stop=891, step=1)
```

```
df.dtypes # Shows the data types of each columns
```

Output Block

```
survived int64
pclass int64
sex object
age float64
sibsp int64
parch int64
fare float64
embarked object
class object
who object
adult_male bool
deck object
embark_town object
alive object
alone bool
family_size int64
dtype: object
```

```
df.nunique()
```

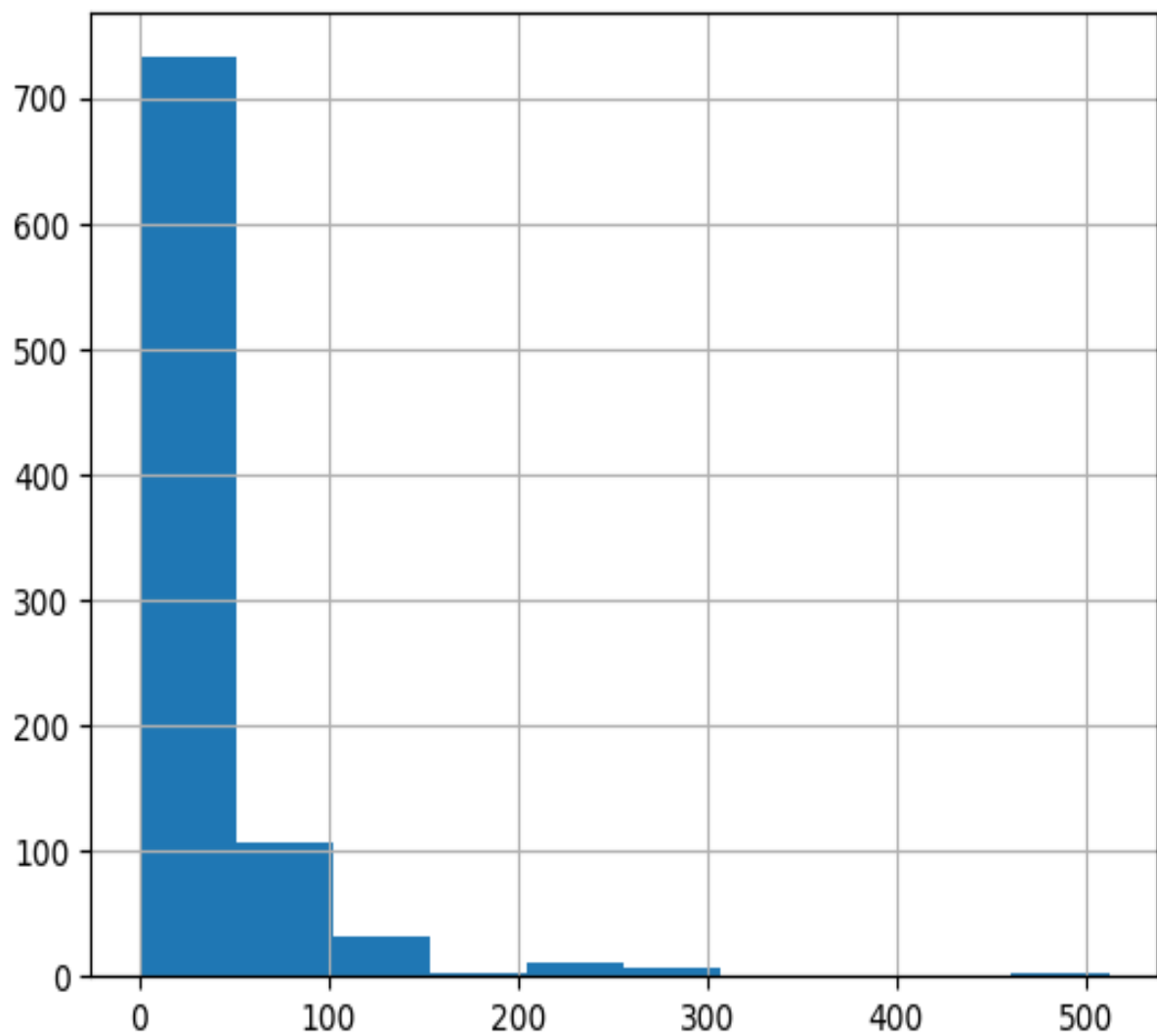
Output Block

```
survived 2
pclass 3
sex 2
age 88
sibsp 7
parch 7
fare 248
embarked 3
class 3
who 3
adult_male 2
deck 7
embark_town 3
alive 2
alone 2
family_size 9
dtype: int64
```

```
df['fare'].hist(bins=10)
```

Output Block

<Axes: >



```
df.values # To display the underlying data in the dataframe it is nothing but numpy array
```

Output Block

```
array([[0, 3, 'male', ..., 'Southampton', 'no', False],
       [1, 1, 'female', ..., 'Cherbourg', 'yes', False],
       [1, 3, 'female', ..., 'Southampton', 'yes', True],
       ...,
       [0, 3, 'female', ..., 'Southampton', 'no', False],
       [1, 1, 'male', ..., 'Cherbourg', 'yes', True],
       [0, 3, 'male', ..., 'Queenstown', 'no', True]],
      shape=(891, 15), dtype=object)
```

```
new_var = df.T #This is to transpose the data into a variable
new_var.head()
```

Output Block

```
0 1 2 3 4 5 6 7 8 \
survived 0 1 1 1 0 0 0 0 1
pclass 3 1 3 1 3 3 1 3 3
sex male female female female male male male male female
age 22.0 38.0 26.0 35.0 35.0 NaN 54.0 2.0 27.0
sibsp 1 1 0 1 0 0 0 3 0

9 ... 881 882 883 884 885 886 887 888 \
survived 1 ... 0 0 0 0 0 0 1 0
pclass 2 ... 3 3 2 3 3 2 1 3
sex female ... male female male male female male female female
age 14.0 ... 33.0 22.0 28.0 25.0 39.0 27.0 19.0 NaN
sibsp 1 ... 0 0 0 0 0 0 0 1

889 890
survived 1 0
pclass 1 3
sex male male
age 26.0 32.0
sibsp 0 0

[5 rows x 891 columns]
```

```
# Sorting the dataframes - Sorting by the column names
new_var = df.sort_index(axis=1, ascending=False)
new_var.head()
```

Output Block

```
who survived sibsp sex pclass parch fare embarked \
0 man 0 1 male 3 0 7.2500 S
1 woman 1 1 female 1 0 71.2833 C
2 woman 1 0 female 3 0 7.9250 S
3 woman 1 1 female 1 0 53.1000 S
4 man 0 0 male 3 0 8.0500 S

embark_town deck class alone alive age adult_male
0 Southampton NaN Third False no 22.0 True
1 Cherbourg C First False yes 38.0 False
2 Southampton NaN Third True yes 26.0 False
3 Southampton C First False yes 35.0 False
4 Southampton NaN Third True no 35.0 True
```

```
# Sorting the dataframes - Sorting by the values
new_var = df.sort_values(by="survived", ascending=False)
new_var.head()
```

Output Block

```

survived pclass sex age sibsp parch fare embarked class \
889 1 1 male 26.0 0 0 30.0000 C First
887 1 1 female 19.0 0 0 30.0000 S First
1 1 1 female 38.0 1 0 71.2833 C First
2 1 3 female 26.0 0 0 7.9250 S Third
3 1 1 female 35.0 1 0 53.1000 S First

```

```

who adult_male deck embark_town alive alone
889 man True C Cherbourg yes True
887 woman False B Southampton yes True
1 woman False C Cherbourg yes False
2 woman False NaN Southampton yes True
3 woman False C Southampton yes False

```

```

# Insert a new column
import random as rand
df['survival_rate'] = rand.randint(1,8)
df.head()

```

Output Block

```

survived pclass sex age sibsp parch fare embarked class \
0 0 3 male 22.0 1 0 7.2500 S Third
1 1 1 female 38.0 1 0 71.2833 C First
2 1 3 female 26.0 0 0 7.9250 S Third
3 1 1 female 35.0 1 0 53.1000 S First
4 0 3 male 35.0 0 0 8.0500 S Third

```

```

who adult_male deck embark_town alive alone survival_rate
0 man True NaN Southampton no False 1
1 woman False C Cherbourg yes False 1
2 woman False NaN Southampton yes True 1
3 woman False C Southampton yes False 1
4 man True NaN Southampton no True 1

```

```

# Slice and dice the data
df.loc[:,['alive', 'age']] # Returns all rows and 2 columns

```

Output Block

```
alive age
0 no 22.0
1 yes 38.0
2 yes 26.0
3 yes 35.0
4 no 35.0
.. ... ..
886 no 27.0
887 yes 19.0
888 no NaN
889 yes 26.0
890 no 32.0
```

```
[891 rows x 2 columns]
```

```
# Slice and dice the data
df.iloc[3:8,1:3]# pulling by index. Similar to indexing in string and arrays etc.
```

Output Block

```
pclass sex
3 1 female
4 3 male
5 3 male
6 1 male
7 3 male
```

```
# Slice and dice the data
df.iloc[0:5, :] # top 5 rows and all the columns
```

Output Block

```
survived pclass sex age sibsp parch fare embarked class \
0 0 3 male 22.0 1 0 7.2500 S Third
1 1 1 female 38.0 1 0 71.2833 C First
2 1 3 female 26.0 0 0 7.9250 S Third
3 1 1 female 35.0 1 0 53.1000 S First
4 0 3 male 35.0 0 0 8.0500 S Third

who adult_male deck embark_town alive alone survival_rate
0 man True NaN Southampton no False 1
1 woman False C Cherbourg yes False 1
2 woman False NaN Southampton yes True 1
3 woman False C Southampton yes False 1
4 man True NaN Southampton no True 1
```

```
# Slice and dice the data
df.iloc[[1,3,5],[0,2,9]] # pulling specific rows and columns
```

Output Block

```

survived sex who
1 1 female woman
3 1 female woman
5 0 male man

```

```

# Finding people with the age greater than 22
df[df.age>=22]

```

Output Block

```

survived pclass sex age sibsp parch fare embarked class \
0 0 3 male 22.0 1 0 7.2500 S Third
1 1 1 female 38.0 1 0 71.2833 C First
2 1 3 female 26.0 0 0 7.9250 S Third
3 1 1 female 35.0 1 0 53.1000 S First
4 0 3 male 35.0 0 0 8.0500 S Third
.. ..
884 0 3 male 25.0 0 0 7.0500 S Third
885 0 3 female 39.0 0 5 29.1250 Q Third
886 0 2 male 27.0 0 0 13.0000 S Second
889 1 1 male 26.0 0 0 30.0000 C First
890 0 3 male 32.0 0 0 7.7500 Q Third

```

```

who adult_male deck embark_town alive alone survival_rate
0 man True NaN Southampton no False 1
1 woman False C Cherbourg yes False 1
2 woman False NaN Southampton yes True 1
3 woman False C Southampton yes False 1
4 man True NaN Southampton no True 1
.. ..
884 man True NaN Southampton no True 1
885 woman False NaN Queenstown no False 1
886 man True NaN Southampton no True 1
889 man True C Cherbourg yes True 1
890 man True NaN Queenstown no True 1

```

```

[510 rows x 16 columns]

```

```

# Creating the dataframes with dates
months = pd.date_range('20170101', periods=10, freq='ME')
months

```

Output Block

```

DatetimeIndex(['2017-01-31', '2017-02-28', '2017-03-31', '2017-04-30',
'2017-05-31', '2017-06-30', '2017-07-31', '2017-08-31',
'2017-09-30', '2017-10-31'],
dtype='datetime64[ns]', freq='ME')

```

```

import random
import numpy as np

```

```
pre_def= pd.DataFrame(np.random.randn(10,5), index = months, columns = list('ABCDE'))  
pre_def
```

Output Block

```
A B C D E  
2017-01-31 0.369567 -0.160100 -0.889719 1.476533 -0.174734  
2017-02-28 -0.655871 0.256547 -1.364289 -1.078702 -0.086989  
2017-03-31 0.623860 -0.296077 2.297316 0.808095 -0.305247  
2017-04-30 0.183064 -0.356909 1.052069 -1.435328 -0.798982  
2017-05-31 -0.788301 1.092098 -1.161520 2.019926 -0.906696  
2017-06-30 0.153568 -0.176740 0.448892 1.287081 -0.400232  
2017-07-31 1.145117 0.905730 0.374236 -1.533891 0.218647  
2017-08-31 -1.017093 -0.429607 0.389950 0.896838 -0.222126  
2017-09-30 0.415799 -0.278718 0.445280 1.073169 0.670445  
2017-10-31 0.416873 -1.479700 0.406721 0.679054 0.069595
```